Msimulizi: Project Brief

Audience: Kiro (spec-driven build). Read sections 1 to 3 before anything else, because they are hard constraints from the Official Rules and override any design preference below.

1. Pitch

Msimulizi ("the narrator") turns a stream of synthetic football events into live, explainable match intelligence. It detects key moments, explains why each one matters, and renders the result as timed overlay cards and radio-style commentary. Narration is written natively in Swahili, with English and French alongside, and every viewer can tailor it: analyst or casual fan, favourite club, a single player to follow, or audio-only for low bandwidth. At full time it produces an automated recap.

The human angle is that millions of fans follow football through Swahili radio commentary. Msimulizi is the digital, personalised equivalent, and it explains the game instead of only reporting it.

2. Hard constraints from the Official Rules

The data must be synthetic and football-realistic. No real Premier League data or match footage may be used as the demo feed, and none is supplied by the organisers. Real open datasets may only be used to calibrate the generator's statistics (see section 6).

The project must be newly created after Registration opened on 29 September 2026. The git history should start after that date, and no earlier code may be reused.

This extends to files, not just code. Do not copy in any pre-existing scripts, notebooks, trained model files (.pkl, .tflite, .onnx or similar) or datasets from earlier personal projects or from other people's repositories, including the earlier xG and video action-recognition repo that was considered (lukorito99/xg-of-shot-attempts). Everything in the repository must be written or trained during the hackathon window. Reference reading is fine, but reimplement from scratch and say so in the README. Never load pickle files from outside sources, because unpickling untrusted files can execute arbitrary code.

Key dates, all Pacific Time, with East Africa Time in brackets:

- Registration closes 20 October 2026, 12:00 pm PT (10:00 pm EAT). Register now, do not wait.
- Submission window is 6 October 9:00 am PT to 27 October 11:59 pm PT (28 October 09:59 EAT).
- Judging runs until 10 November 2026. The project must stay live and free to test, with no restrictions, until then.

- Internal feature freeze is 24 October and internal submit date is 26 October.

The submission needs the following:

- A newly built project that fits at least one hackathon category.
- A short pitch that names the Microsoft and Azure technologies used, what it does, and what problem it solves.
- A demo video under 2 minutes, hosted publicly on YouTube, Vimeo, Facebook Video or Youku. Judges are not required to watch past 2 minutes.
- A public GitHub repository.
- A working link for testing, with credentials in the instructions if it is private.

Everything must be in English or come with an English translation. This matters a great deal for us. The video, pitch and testing instructions must be in English, and every Swahili line shown on screen needs an English gloss or subtitle, because most judges cannot read Swahili.

The demo video must not contain third-party trademarks or copyrighted music. Use fictional clubs, fictional players, invented badges and kits, and no real broadcaster or league branding.

Content must not be political, offensive, violent, or promote alcohol, drugs or tobacco.

No team member's personal data may appear in the submission or the repository. Never commit API keys or connection strings.

Any third-party SDK, library or dataset must be used within its licence, and open-source use must build on the underlying software, not just repackage it.

3. Scope

In scope are all five pipeline stages: ingest (a), interpret (b), explain (c), render (d) and personalise (e). Also in scope are narrative generation (iii) and multi-language storytelling (vi).

Three further areas come along by implication and should be named in the pitch. Real-time match intelligence (ii) is covered by the rolling stats state, player name tagging, and basic pass distance and accuracy. Explainability (iv) is covered by the reason attached to every key moment. Data innovation (v) is covered by the synthetic match generator.

Out of scope are auto-eventing from video (including any video action-recognition model), ball and shot speed metrics, and pass difficulty ratings. The brief only asks teams to consider these. Video auto-eventing also needs real footage, which the rules do not supply and which conflicts with the synthetic-data requirement. It may appear as a single roadmap line in the README, but it must not be presented as a feature.

4. How we will be judged

Stage one is a pass/fail check that the project fits the theme and reasonably uses the featured Microsoft technologies. Stage two scores five equally weighted criteria at 20% each. Ties are broken by Technological Implementation first, then the next criterion in order.

Criterion	What we must show
Technological Implementation	A creative, well-validated synthetic data generator, clean documented code, tests, and effective use of hero technologies (Microsoft Foundry, Agent Framework, Azure MCP, Fabric, Azure AI Services, Azure databases, GitHub Copilot agent mode).
Agentic Design and Innovation	Real multi-agent orchestration with distinct roles, shared state, handoffs and failure recovery. A plain pipeline will score poorly here, see section 5.
Real-World Impact and Applicability	A credible path to studio, broadcast and streaming use, and genuine value for Swahili-speaking audiences.
User Experience and Presentation	An intuitive viewer UI, a balanced frontend and backend, and a tight 2-minute video.
Adherence to Hackathon Category	Explicit alignment with the category descriptions we target.

Category prizes are Best Use of Microsoft Foundry, Best Enterprise Solution, Best Multi-Agent System, and Best Azure Cloud Native Integration. A project can win one Grand Prize and up to one Category Prize. Our primary targets are the Grand Prize and Best Multi-Agent System, with Foundry as a close secondary. Judges may reassign a project to a different category.

5. Agent design

The original idea was a linear pipeline. To score on agentic design, build it as cooperating agents on Microsoft Agent Framework, each with a narrow role and all sharing one match state store.

- The Interpreter agent consumes events and maintains the rolling match state (possession, field tilt, shots and xG, pressure, momentum, tempo).
- The Moment agent detects key moments from the state and ranks them.
- The Explainer agent attaches a short reason to each moment, citing the event IDs that justify it.
- The Narrator agent writes the commentary natively in the requested language.

- The Personaliser agent applies the viewer profile (mode, club, player focus, language, metric focus) to selection and phrasing.
- The Verifier agent checks every narration line against the structured events and rejects anything ungrounded.
- The Overlay Renderer is deterministic code, not an LLM. It converts approved lines into timed JSON cards.

Failure recovery is the part judges will probe. If the Verifier rejects a line, the Narrator retries once with the rejection reason. If that fails, a deterministic template line is used so the feed never stalls. If the model call is slow or fails, the system falls back to cached or templated narration and logs the event. Show this once in the demo video or the repository README.

Expose the match state through an MCP server so other tools or agents can query it ("what is the momentum in the last 10 minutes?"). This is a cheap way to show MCP integration.

6. Data: synthetic generator and sourcing

We do not source match data. We generate it. The assumption that we need real data from somewhere conflicts with the rules, and scraping league data (Premier League, FBref, Understat and similar) risks disqualification and licence problems.

Real open data has one legitimate use: calibrating the generator so its statistics look football-realistic. Download it locally, compute aggregate parameters, and discard it. Never commit, ship or replay the raw data. Extract only figures such as pass completion by pitch zone, shot location and xG distribution, events per minute, and pressure rates. Include a short DATA_PROVENANCE.md explaining exactly what was used and how.

Sources worth using, all free (full links are in section 19):

- StatsBomb Open Data is the best fit. It is event-level JSON with shot locations, pass outcomes, pressure events and some 360 freeze-frame data, and the statsbombpy package loads it without authentication. Its terms require you to state the source as StatsBomb and use their logo if you publish or share anything based on the data. Because the demo video must not contain third-party trademarks, put the attribution and logo in the README and the app's About page only, never in the video, and keep the video free of any StatsBomb-derived charts. Read their user agreement before relying on it.
- The Wyscout public match event dataset (Pappalardo et al., Scientific Data 2019, hosted on figshare) covers a full season of seven competitions. Cite the paper if used, and check the licence on the figshare page. It includes the 2017/18 Premier League, so calibrate only from the other competitions in it. The rules say the project must use synthetic data and no real Premier League data is supplied, so avoid even the appearance of building on Premier League data.

- Metrica Sports sample data offers anonymised, synchronised tracking and event data for three games. It is useful for tempo and spacing sanity checks, but it is small and the use is "responsible use" with no obvious licence, so check the repository before depending on it. Tracking data is not needed for this project.

Do not scrape FBref, Understat, WhoScored, Transfermarkt or the Premier League site. Their terms generally prohibit it and it would undermine the synthetic-data requirement.

Build the generator as a standalone, seeded, reproducible module. Inputs are two fictional squads and a scripted storyline. Storylines to build first are a comeback from two goals down, a red card that shifts control, a late equaliser, and a dominant side that loses. Scripted storylines make the demo reliable and give the narrator real drama to explain.

Calibration targets are approximate and should be sanity-checked against the open data. A typical match has roughly 800 to 1,000 passes in total, with completion around 80 to 85 percent, around 20 to 30 shots in total, and an average near 2.5 to 3 goals.

Each event should carry at least the following fields: event_id, match_id, period, minute, second, team, player, event_type, start and end coordinates on a 0 to 100 pitch, outcome, xg for shots, pressure flag, and the running score. Event types include pass, shot, tackle, interception, possession change, pressure event, foul, card, substitution and goal. Also emit match metadata (teams, squads, formation, kickoff) as the first event.

Add a validation script that compares generated statistics with the calibration targets and fails if they drift. This is concrete evidence for the "optimized synthetic data creation" judging question.

Use fictional clubs and players. Names can be East African in flavour, but must not match real current players or clubs.

7. Interpretation and explainability

Keep the rolling state light. Track possession share, field tilt (share of passes in the final third), shots and cumulative xG, pressure events per minute, a simple momentum score over a sliding window, and tempo (events per minute).

Expected goals (xG) is computed by one function that the generator and the state tracker both call, so every shot carries a consistent value. Implement it fresh as a small logistic regression on shot distance and angle, plus simple flags for header and penalty, with coefficients tuned against shot statistics from the open data (not copied from any earlier model). Keep it in plain Python with unit tests, and expose it behind a single interface so a heavier model could replace it later. A more complex gradient-boosted model is not worth the time, because the shots are synthetic and the explanations only need xG to be sensible and consistent.

Key moments include goals, red cards, big chances, momentum swings, lead changes, milestones (first shot, hat-trick, a player's tenth pass into the box), and late-game turning points.

Every moment must carry a reason and its evidence. Example reasons are "three high-pressure turnovers in four minutes handed the initiative to the away side", "the home team has lost midfield control, with field tilt down from 62 to 41 percent", and "the equaliser at 88 minutes changes the rhythm of the match". Use plain language for casual fans and the underlying numbers for analysts. The reason is generated from the state, never invented, and the cited event IDs are stored with it so the Verifier and the UI can show the evidence.

Frame explanations around control versus chaos, tactical pressure changes and rhythm, because the brief names these explicitly.

8. Narration and languages

Write Swahili natively with few-shot examples of real radio commentary style, rather than writing English and translating. Use Azure AI Translator only for the other languages if needed, or generate French natively as well. Compare models in Foundry on Swahili quality early and pick the best.

Ground the model on structured events only. It may narrate minute, players, event type, score state and the supplied reason, and nothing else. A wrong scorer in a live narrator is an instant credibility loss.

Narrate key moments only, not every event. Target a latency from event to caption under about 2 seconds and to audio under about 5 seconds. Pre-generate audio for common phrases if needed.

Check early that Azure Speech has Swahili voices good enough for the pitch. Voices exist for sw-KE and sw-TZ, but verify the current list and listen before committing. If quality is weak, make captions the primary output and audio the enhancement.

Build a quick native-speaker rating and correction loop into the UI (thumbs and an edit box), store the corrections, and feed good ones back as few-shot examples. Show this loop briefly in the demo.

Every Swahili output shown to judges needs an English line beneath it.

9. Rendering and overlay output

Each approved narration or insight is emitted as machine-readable JSON, timed against the match clock, so it could drive graphical overlays on a live feed. The organisers mention a rendering partner but give no schema, so define our own and document it.

Suggested fields: card_id, match_clock (period, minute, second), display_from, display_duration_ms, card_type (goal, key_moment, stat, player_spotlight, recap), priority, player and team references, headline and body text keyed by language, the reason, evidence_event_ids, and a stat payload where relevant. Keep text within safe lengths for lower-third graphics.

The viewer UI renders these cards beside a simple match visual (pitch with ball or event dots, score, clock). One clean on-screen card design is enough. The UI should consume the same JSON a broadcast overlay would, which proves the output is production-shaped.

10. Personalisation

The viewer profile has these dimensions, all applied by the Personaliser agent over the same underlying intelligence:

- Mode: analyst (numbers, xG, tilt, evidence) or casual fan (short, emotional, plain language).
- Language: Swahili, English or French.
- Favourite club: shifts loyalty, emphasis and which moments are surfaced.
- Player focus: follows one player through the match with a running spotlight.
- Single metric focus: for example only pressure or only xG.
- Audio-only radio mode: low-bandwidth, captions optional.

The grand prize text mentions accessible experiences, so treat accessibility as a feature. Include captions, an audio-only mode, readable contrast, keyboard navigation and a simple-language option.

The analyst and casual views of the same moment should look clearly different side by side in the demo. This is the most visible proof of stage (e).

11. Architecture and Azure stack

Design the event source as a swappable adapter so we can move from a local file replay to Azure Event Hubs to Fabric Eventstream without touching the agents. This protects us from Fabric setup risk while keeping Fabric as a stretch goal.

Suggested stack:

- Microsoft Foundry for models, agent hosting and evaluations.
- Microsoft Agent Framework for orchestration.
- Azure Event Hubs for the live stream, with Fabric Eventstream and a KQL database as the optional upgrade.
- Azure Functions for event-driven steps and Azure Container Apps for the API.

- Azure AI Speech for text to speech, and Azure AI Translator where used.
- An Azure database (Cosmos DB or similar) for match state, moments and corrections.
- A Next.js or similar frontend on Container Apps or Static Web Apps.
- GitHub Actions for CI, and optionally GitHub Copilot agent mode for parts of development, mentioned in the pitch if used.

Support a replay mode that streams a generated match at real speed, or accelerated for testing. A live match will not line up with judging time, so replay is the demo path.

Keep Azure costs capped. The project must stay available and free to test until 10 November, so set budgets, rate limits and caching.

12. Responsible AI and enterprise readiness

Label all content clearly as synthetic and AI-generated. Use Azure AI Content Safety on model output. Log every narration with its evidence and Verifier verdict, which gives an audit trail. Use Foundry evaluations for groundedness. Keep configuration, secrets and environments separate (Key Vault, environment variables), and document deployment. These points address the Enterprise and Cloud Native category descriptions.

13. Evaluation and quality bar

Track and report these numbers in the README and pitch:

- Groundedness: percentage of narration lines whose every claim traces to supplied events.
- Latency: p50 and p95 from event to caption and to audio.
- Swahili quality: native-speaker ratings over a sample of lines.
- Determinism: the same seed and storyline reproduce the same match.
- Failure recovery: the fallback path triggers correctly under simulated model failure.

Include unit tests for the generator, state aggregation and moment detection, and an integration test that replays a full match end to end.

14. Demo and submission plan

The video is under 2 minutes, so plan every second. A workable storyboard:

- 0:00 to 0:15, the problem and the Swahili radio angle.
- 0:15 to 0:45, a replayed match with live Swahili captions, English gloss, audio and overlay cards.
- 0:45 to 1:05, a key moment with its "why it matters" reason and evidence.
- 1:05 to 1:30, the analyst versus casual toggle, favourite club and player focus.
- 1:30 to 1:45, the agents and failure recovery, with the Verifier rejecting a line.

- 1:45 to 2:00, full-time recap, Azure stack summary and impact.

Provide a public demo URL that needs no login, defaults to a bundled replay match, and works even if model quota runs out by using cached narration. Write clear testing instructions in English. Judges may judge from the text and video alone, so those two must stand on their own.

The pitch text should name every Microsoft and Azure technology actually used, state what the project does and what problem it solves, and explicitly mention the areas covered: real-time intelligence, explainability, synthetic data innovation, narrative generation and multi-language storytelling.

15. Timeline

Dates (2026)	Milestone
3 to 5 Oct	Register on aka.ms/insidethegame, Azure and Foundry setup, repo, event schema, Swahili voice check.
6 to 10 Oct	Match generator with scripted storylines, validation script, replay streamer.
11 to 14 Oct	Rolling state, moment detector, explanations with evidence.
15 to 18 Oct	Agents, native Swahili narration, Verifier, text to speech.
19 to 21 Oct	Overlay JSON, viewer UI and card, personalisation modes.
22 to 23 Oct	English and French, evaluations, correction loop, deployment hardening.
24 to 25 Oct	Feature freeze, video, README, pitch.
26 Oct	Submit with buffer.
27 Oct, 11:59 pm PT	Hard deadline (28 Oct 09:59 EAT).

16. Risks and mitigations

- Weak Swahili generation or voice. Mitigate with early model comparison, few-shot examples, native review, and captions as the primary output.
- Fabric or Eventstream setup consuming time. Mitigate with the swappable source adapter and Event Hubs as the default.
- Latency making audio feel late. Mitigate by narrating key moments only and caching phrases.

- Hallucinated details. Mitigate with structured-only input and the Verifier agent.
- Demo failing on judging day. Mitigate with replay mode, cached output, and cost caps.
- Scope creep. Mitigate by finishing a thin end-to-end path first (one replayed event, one Swahili line, one audio clip, one card), then widening.
- Rules violations. Mitigate with synthetic-only data, fictional names, no third-party trademarks or music in the video, and English translations everywhere.

17. Definition of done

- A reproducible generator produces at least four scripted storylines, and the validation script passes.
- A replayed match produces Swahili, English and French narration, each line with a reason and evidence.
- The Verifier blocks ungrounded lines and the fallback keeps the feed running.
- The UI shows overlay cards driven by the JSON output, with analyst/casual, club, player focus and audio-only modes working.
- A native-speaker correction loop is working and demonstrated.
- The full-time recap is generated automatically.
- DATA_PROVENANCE.md exists, no raw open data or pre-existing model files are in the repository, and required attributions are in the README and About page.
- The public demo, public repo, English README, pitch and sub-2-minute video are complete and verified against section 2.

18. Open decisions for the team

- Team size and who is the Representative who submits.
- Whether Azure subscription and Foundry access are already available.
- Which category prizes to declare on the submission form beyond the Grand Prize.
- Whether to attempt Fabric Eventstream or stay on Event Hubs.

19. Links and references

Data sources for calibration only (see section 6 for rules of use):

- StatsBomb Open Data: https://github.com/statsbomb/open-data
- statsbombpy loader: https://github.com/statsbomb/statsbombpy
- StatsBomb resource centre and user agreement: https://www.statsbomb.com/resource-centre
- Wyscout public match event dataset (figshare): https://figshare.com/collections/Soccer_match_event_dataset/4415000
- Paper to cite for the Wyscout data: https://www.nature.com/articles/s41597-019-0247-7

- Metrica Sports sample data: https://github.com/metrica-sports/sample-data

Helpful libraries (check each licence before use):

- kloppy, for reading and standardising football event data: https://github.com/PySport/kloppy
- mplsoccer, for pitch plots during development and validation: https://github.com/andrewRowlinson/mplsoccer

Microsoft and hackathon references:

- Hackathon website and submission page: https://aka.ms/insidethegame
- Azure AI Speech language and voice support: https://learn.microsoft.com/azure/ai-services/speech-service/language-support
- Microsoft Agent Framework: https://github.com/microsoft/agent-framework

Swahili voices to audition first, all Azure neural voices: sw-KE-ZuriNeural, sw-KE-RafikiNeural, sw-TZ-RehemaNeural and sw-TZ-DaudiNeural. Confirm they are available in your chosen Azure region before committing.

Not to be used: lukorito99/xg-of-shot-attempts and any of its model or data files, for the reasons in section 2.